

Pointer-Based Array Rotation

Here is analyzing a log of sensor readings stored in an array. For a particular experiment, she wants to re-align the data so that a certain reading becomes the new starting point, but she doesn't want to allocate a new array because memory is tight. Stuart suggests "just rotate it" to the right by K positions. Tata insists it should be done **in-place** and wants to ensure you understand pointer arithmetic, not just array indexing. Your task is to rotate the array to the right by K positions in-place, using only pointer arithmetic (no `[]` indexing).

Problem Description

Given an array of N integers and an integer K , rotate the array to the right by K positions. You must treat the array as dynamically allocated and manipulate it using pointers.

- Rotation to the right by K means: element at position i (0-based) moves to position $(i + K) \bmod N$.

Input Format

- Line 1: N K
- Line 2: N integers $A[0], A[1], \dots, A[N - 1]$.

Output Format

Print the rotated array as N integers separated by spaces.

Constraints

- $1 \leq N \leq 100$
- You must perform the rotation in-place (using only one array).
- Use pointer arithmetic instead of $A[i]$ style indexing.

Sample Input 1 (standard input)

```
5 1
1 2 3 4 5
```

Sample Output 1 (standard output)

```
5 1 2 3 4
```

Sample Input 2 (standard input)

```
7 3  
1 2 3 4 5 6 7
```

Sample Output 2 (standard output)

```
5 6 7 1 2 3 4
```

